

CHƯƠNG 5

QUẢN TRỊ WEB SERVER TỰ ĐỘNG & PROXY

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Giải thích được cách hoạt động của Reverse Proxy.
- Triển khai giao diện quản trị tập trung (Nginx Proxy Manager).
- Cấu hình SSL tự động với Let's Encrypt.
- Áp dụng IaC để version và deploy file cấu hình Nginx.
- Trỏ tên miền thực tế về server và debug proxy (nếu có domain).

5.1 Cơ chế hoạt động của Reverse Proxy

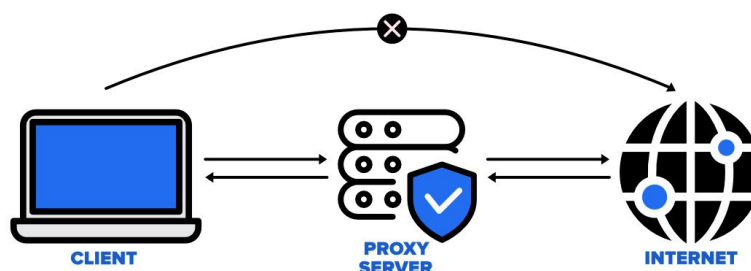
5.1.1 Thế nào là Proxy server

Proxy Server (máy chủ ủy nhiệm) là một máy chủ trung gian nằm giữa các máy khách (client) và máy chủ đích (origin server). Thay vì cho phép thiết bị người dùng giao tiếp trực tiếp với máy chủ đích, Proxy Server sẽ tiếp nhận yêu cầu, xử lý và chuyển tiếp (hoặc từ chối) yêu cầu đó.

Về bản chất, Proxy hoạt động như một lớp trừu tượng hóa (abstraction layer) trong kiến trúc mạng, giúp tách biệt môi trường người dùng cuối với môi trường dịch vụ đích. Điều này mang lại nhiều lợi ích quan trọng trong quản trị hệ thống và an ninh mạng:

- Tăng cường bảo mật
- Kiểm soát và giám sát lưu lượng
- Tối ưu hóa hiệu suất
- Ẩn giấu thông tin nội bộ

Nguyên lý hoạt động cơ bản của một proxy server đó là: Khi một máy khách gửi yêu cầu truy cập tài nguyên (ví dụ: truy cập website), thì yêu cầu được gửi đến Proxy Server thay vì trực tiếp đến máy chủ đích.



Hình 5.1: Nguyên lý hoạt động của Proxy server

Khi đó, Proxy Server phân tích yêu cầu, áp dụng các chính sách đã được cấu hình (access control list, content filtering, authentication...). Nếu yêu cầu hợp lệ, Proxy sẽ thay mặt client thiết lập kết nối với máy chủ đích. Proxy nhận phản hồi từ máy chủ đích và chuyển trả về cho client.

Quá trình này cho phép Proxy kiểm soát, ghi log, sửa đổi hoặc lưu cache nội dung trước khi trả về người dùng.

Phân loại Proxy Server

a. Forward Proxy (Proxy hướng tới client)

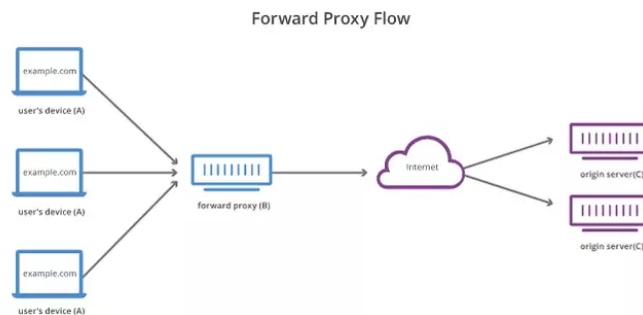
Đây là loại Proxy phổ biến nhất trong môi trường doanh nghiệp và tổ chức.

Vị trí: Đứng về phía client.

Chức năng chính:

- Kiểm soát truy cập internet của người dùng nội bộ.
- Lọc nội dung (web filtering).
- Ẩn địa chỉ IP thực của client.
- Ghi log hoạt động truy cập.

Ví dụ thực tế: Trong mạng nội bộ của một công ty, khi nhân viên cố gắng truy cập Facebook hoặc TikTok, yêu cầu sẽ được chuyển đến Forward Proxy của công ty. Proxy kiểm tra chính sách, nếu bị cấm thì từ chối ngay lập tức. Ngược lại, Proxy sẽ thay mặt gửi yêu cầu đến Facebook và trả kết quả về cho nhân viên. Toàn bộ quá trình này hoàn toàn minh bạch với người dùng cuối (transparent) hoặc có thể yêu cầu cấu hình thủ công trên trình duyệt.



Hình 5.2: Mô hình của Forward proxy

Ứng dụng trong quản trị:

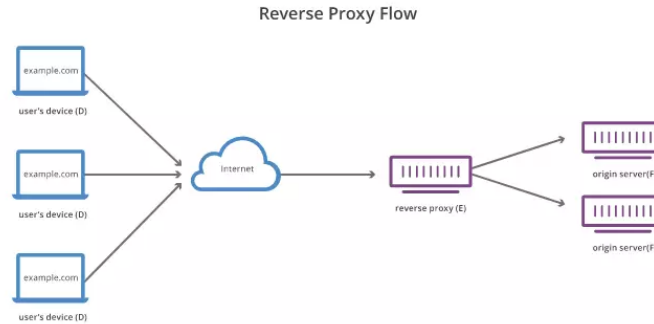
- Thực thi Acceptable Use Policy (AUP)
- Giám sát và điều tra sự cố an ninh
- Tiết kiệm băng thông qua cơ chế caching

b. Reverse Proxy (Proxy hướng tới server)

Đây là loại Proxy cực kỳ quan trọng trong triển khai hệ thống phần mềm quy mô lớn.

Vị trí: Đứng trước các máy chủ ứng dụng (web server, application server).

Chức năng chính:



Hình 5.3: Mô hình hoạt động của Reverse proxy

- **Cân bằng tải (Load Balancing):** Một trang web phổ biến nhận hàng triệu lượt truy cập mỗi ngày thường không thể xử lý toàn bộ lưu lượng bằng một máy chủ gốc (origin server) duy nhất. Thay vào đó, trang web có thể được phân phối trên một nhóm (pool) các máy chủ khác nhau, tất cả cùng xử lý các yêu cầu cho cùng một dịch vụ. Lúc này, Reverse Proxy đóng vai trò cung cấp giải pháp cân bằng tải, phân phối lưu lượng truy cập đến một cách đồng đều giữa các máy chủ để tránh tình trạng bất kỳ máy chủ nào bị quá tải. Nếu một máy chủ bị hỏng hoàn toàn, các máy chủ còn lại sẽ tự động tiếp nhận và xử lý lưu lượng, đảm bảo tính sẵn sàng và liên tục của hệ thống.
- **Tăng cường bảo mật** (ẩn giấu cấu trúc server backend, ngăn chặn tấn công trực tiếp): Khi sử dụng Reverse Proxy, website hoặc dịch vụ không cần tiết lộ địa chỉ IP thực của các máy chủ gốc. Điều này khiến cho các cuộc tấn công có chủ đích (targeted attack) trở nên khó khăn hơn nhiều, chẳng hạn như tấn công DDoS. Kẻ tấn công chỉ có thể nhắm đến Reverse Proxy (ví dụ: hạ tầng CDN của Cloudflare), vốn được trang bị các biện pháp bảo mật chặt chẽ hơn và có nhiều tài nguyên để chống chịu các cuộc tấn công mạng.
- **Cân bằng tải toàn cầu (Global Server Load Balancing - GSLB):** Đây là hình thức cân bằng tải nâng cao, cho phép website được triển khai trên nhiều máy chủ đặt tại các khu vực địa lý khác nhau trên thế giới. Reverse Proxy sẽ tự động chuyển hướng người dùng đến máy chủ gần nhất về mặt địa lý. Nhờ đó, khoảng cách truyền dữ liệu giữa yêu cầu và phản hồi được rút ngắn đáng kể, giúp giảm thời gian tải trang (latency) và cải thiện trải nghiệm người dùng.
- **SSL Termination** (giải mã TLS tại Proxy): Việc mã hóa và giải mã các kết nối SSL/TLS cho từng client có thể tiêu tốn rất nhiều tài nguyên tính toán của máy chủ gốc. Reverse Proxy có thể được cấu hình để đảm nhận nhiệm vụ giải mã tất cả các yêu cầu đến và mã hóa lại các phản hồi đi ra, giúp giải phóng tài nguyên quý giá cho các máy chủ ứng dụng phía sau, từ đó tăng hiệu suất tổng thể của hệ thống.
- **Compression, Rate limiting:** Reverse Proxy có khả năng lưu cache nội dung, từ đó cải thiện đáng kể hiệu suất hệ thống. Ví dụ: Một người dùng tại Paris truy cập vào website có máy chủ gốc đặt tại Los Angeles. Người dùng sẽ được kết nối với Reverse Proxy đặt tại Paris. Proxy này sẽ gửi yêu cầu đến máy chủ gốc tại Los Angeles, nhận phản hồi và lưu cache lại. Những người dùng Paris sau đó sẽ nhận được dữ liệu từ bản cache cục bộ trên Reverse Proxy Paris, giúp tốc độ truy cập

nhANH hơn rất nhiều.

Ví dụ phổ biến: Nginx, HAProxy, Apache Traffic Server, Cloudflare, AWS Application Load Balancer.

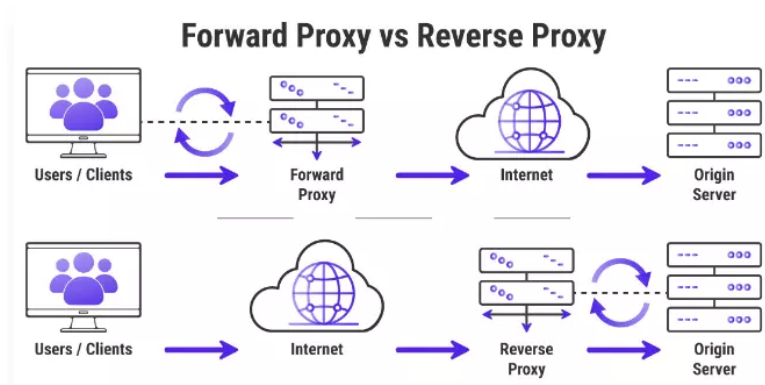
Lợi ích nổi bật trong triển khai hệ thống:

- Cho phép triển khai kiến trúc microservices an toàn
- Dễ dàng scale horizontally
- Bảo vệ backend servers khỏi DDoS và các tấn công layer 7

c. Sự khác nhau giữa reverse proxy với forward proxy

Một reverse proxy là một máy chủ đặt trước một hoặc nhiều máy chủ web, chặn các yêu cầu từ máy khách. Điều này khác với forward proxy, nơi proxy đặt trước máy khách. Với reverse proxy, khi máy khách gửi yêu cầu đến máy chủ gốc của một trang web, những yêu cầu đó sẽ bị chặn ở biên mạng bởi reverse proxy server. Sau đó, reverse proxy server sẽ gửi yêu cầu đến và nhận phản hồi từ máy chủ gốc.

Sự khác biệt giữa reverse proxy và forward proxy là nhỏ nhưng quan trọng. Một cách đơn giản để tóm tắt nó sẽ là nói rằng một forward proxy nằm trước một máy client và đảm bảo rằng không có máy chủ gốc nào giao tiếp trực tiếp với máy client cụ thể đó. Mặt khác, reverse proxy nằm trước máy chủ gốc và đảm bảo rằng không có máy client nào giao tiếp trực tiếp với máy chủ gốc đó.



Hình 5.4: Sự khác biệt giữa hai loại Proxy

5.1.2 Cơ chế làm việc của Reverse Proxy

Reverse proxy giúp chặn các yêu cầu từ Internet, sau đó định tuyến và chuyển tiếp chúng đến các máy chủ thực hiện nhiệm vụ xử lý.

Quá trình xử lý một yêu cầu (request) diễn ra tuần tự qua các bước sau:

- **Bước 1 – Tiếp nhận yêu cầu:** Người dùng truy cập vào ứng dụng/website. Yêu cầu này không đi trực tiếp đến máy chủ gốc mà sẽ đi thẳng tới Reverse Proxy.
- **Bước 2 – Xử lý bảo mật & Lọc:** Reverse proxy kiểm tra yêu cầu (ví dụ: chặn IP độc hại, lọc dữ liệu rác, ngăn chặn tấn công DDoS). Các thao tác bao gồm:
 - Giải mã SSL/TLS (SSL Termination)
 - Kiểm tra xác thực (Authentication)

- Áp dụng các quy tắc bảo mật: Rate limiting, WAF (Web Application Firewall), lọc request độc hại
- Phân tích URL, Host header để quyết định chuyển tiếp đến backend nào
- **Bước 3 – Chuyển tiếp (Routing/Load Balancing):** Nếu yêu cầu hợp lệ, Reverse proxy sẽ quyết định chuyển tiếp nó đến máy chủ nội bộ nào (đối với hệ thống có nhiều máy chủ) dựa trên các thuật toán cân bằng tải.
- **Bước 4 – Nhận phản hồi:** Máy chủ nội bộ xử lý xong sẽ gửi dữ liệu trả về cho Reverse Proxy.
- **Bước 5 – Trả kết quả cho Client:** Reverse Proxy gửi dữ liệu phản hồi này về cho người dùng, đóng vai trò như người duy nhất giao tiếp với client (client không biết địa chỉ thực của các máy chủ nội bộ). Ngoài ra có thể thực hiện thêm các thao tác như: Lưu cache (nếu được cấu hình), Nén dữ liệu (Compression), Mã hóa SSL/TLS trước khi gửi về client.

Một số giải pháp Reverse proxy phổ biến bao gồm:

- Nginx (rất phổ biến)
- HAProxy
- Apache Traffic Server
- Traefik, Envoy (Kubernetes)
- Cloudflare, AWS ALB/ELB, Azure Application Gateway

Yêu cầu đối với sinh viên: Hãy tìm hiểu nhanh các giải pháp Reverse proxy và so sánh tính năng, ưu/nhược điểm của mỗi loại.

5.1.3 Bảo mật ứng dụng web với proxy

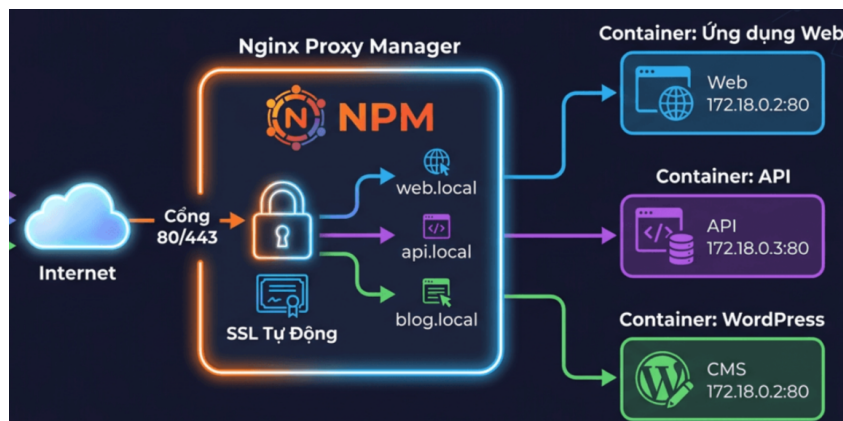
Với các ứng dụng web được triển khai trên môi trường thực tế thì cần có nhiều phương án phối hợp để có thể đảm bảo an toàn thông tin cho hệ thống trước các cuộc tấn công đa dạng bên ngoài. Quản trị viên hệ thống có thể triển khai các giải pháp bảo vệ lớp máy chủ với các hệ thống phát hiện xâm nhập, bảo vệ tầng vật lý, bảo vệ lớp 2 tránh tấn công local. Ngoài ra cũng nên gia cố hệ thống với việc tích hợp tường lửa web (WAF) ở vòng ngoài, bên cạnh đó, việc sử dụng proxy cũng giúp cho việc che dấu thông tin bên trong với thế giới bên ngoài.

Nguyên tắc cốt lõi đó là sử dụng Reverse Proxy như “Lớp Bảo vệ Tiên tiến”. Theo đó, Reverse Proxy đóng vai trò là điểm vào duy nhất (Single Entry Point) của toàn bộ ứng dụng web. Thay vì để người dùng truy cập trực tiếp vào máy chủ ứng dụng (backend), tất cả lưu lượng phải đi qua Reverse Proxy. Nhờ đó, Proxy trở thành lớp bảo mật mạnh mẽ, thực thi nhiều cơ chế bảo vệ trước khi yêu cầu đến được backend.

Các kỹ thuật bảo mật phổ biến gồm có:

- **Kết hợp với WAF:** Giải pháp thường được triển khai hiện nay đó là tích hợp trực tiếp WAF trên hệ thống Reverse proxy, điển hình đó là việc kết hợp Mod_security (giải pháp tường lửa web mã nguồn mở mạnh mẽ và rất phổ biến) trên Nginx/Apache, đồng thời tích hợp bộ quy tắc cốt lõi **OWASP CRS**, đây là một tập hợp các quy tắc phát hiện tấn công chung được thiết kế cho các WAF như ModSecurity và Coraza. Nó bảo vệ các ứng dụng web khỏi các mối đe dọa như SQL Injection (SQLi) và Cross-Site Scripting (XSS) bằng cách sử dụng Chấm điểm bất thường (Anomaly Scoring) thay vì chỉ chặn các sự kiện riêng lẻ.

- **Ẩn giấu cấu trúc Backend (IP Hiding & Topology Obfuscation):** Hệ thống backend servers không cần public IP, chỉ giao tiếp nội bộ với Reverse Proxy. Như vậy kẻ tấn công chỉ nhìn thấy IP của Proxy dẫn đến rất khó thực hiện tấn công trực tiếp vào máy chủ web của dịch vụ, cũng như các dịch vụ khác được đặt phía sau của hệ thống Proxy.



Hình 5.5: Sơ đồ hoạt động của proxy server

Sơ đồ triển khai hệ thống như trên hình cho thấy, proxy server sẽ đóng vai trò che giấu thông tin cho toàn bộ các máy chủ dịch vụ: web, API server, CMS website bên trong nội bộ. Toàn bộ giao tiếp từ người dùng bên ngoài sẽ được kết nối và truy xuất đến proxy, sau đó proxy sẽ xử lý truy vấn và phân phối về các nhánh (các sub domain được cấu hình bên trong), từ đó sẽ chuyển tới các máy chủ dịch vụ đích đặt phía sau để xử lý. Như vậy việc xử lý phía trong mạng nội bộ hoàn toàn transparent với thế giới bên ngoài.

- **Quản lý mã hóa và hủy bỏ SSL/TLS:** Proxy đảm nhận việc giải mã HTTPS, kiểm tra certificate, sau đó mã hóa lại khi chuyển tiếp đến backend. Đồng thời hỗ trợ HTTP/2, HTTP/3 và TLS 1.3 dễ dàng.
- **Quản lý giới hạn tốc độ và điều tiết:** Proxy có thể giới hạn số lượng request theo IP, theo User-Agent, theo Session. Ngoài ra còn giúp chống brute force, credential stuffing (hình thức tấn công mà hacker sử dụng việc nhồi nhét các thông tin đăng nhập, tức là lấy danh sách username + password bị leak ở chỗ này, rồi ”nhồi” vào chỗ khác để xem có ai dùng chung mật khẩu không), API abuse (là hành vi sử dụng sai mục đích, khai thác điểm yếu của các API (Application Programming Interface) để thực hiện tấn công, trích xuất dữ liệu, hoặc gây hại cho hệ thống. API Abuse đang trở thành một trong những vector tấn công phổ biến và nguy hiểm nhất hiện nay).

5.2 Triển khai quản trị tập trung

5.2.1 Giới thiệu về Nginx Proxy Manager (NPM)

Nginx Proxy Manager (NPM) là một ứng dụng chạy trên nền Docker giúp bạn dễ dàng cấu hình Reverse Proxy cho các dịch vụ mạng cài đặt trong homelab hoặc VPS. Nó còn được tích hợp công cụ tạo chứng chỉ SSL miễn phí từ Let's Encrypt.

Tính năng của NPM:

- Giao diện quản lý bảo mật và thân thiện dựa trên Tabler

- Dễ dàng cấu hình chuyển tiếp, chuyển hướng tên miền mà không cần biết gì về cách cấu hình Nginx.
- Tạo chứng chỉ SSL miễn phí từ Let's Encrypt
- Tạo danh sách truy cập và thiết lập xác thực HTTP bảo mật cho máy chủ
- Cấu hình Nginx nâng cao tùy theo nhu cầu sử dụng
- Quản lý tài khoản, phân quyền và lịch sử làm việc.

Thay vì phải viết file cấu hình Nginx thủ công, người quản trị có thể:

- Thêm domain.
- Trỏ domain đến service nội bộ.
- Bật HTTPS.
- Tạo chứng chỉ SSL.
- Cấu hình Access List.
- Quản lý redirection.
- Quản lý stream TCP/UDP proxy.

Tất cả được thực hiện qua giao diện web một cách dễ dàng, ngoài ra, NPM có thể triển khai và chạy bằng Docker, thời gian deploy chỉ trong 2 phút nên không ảnh hưởng hệ thống host.

Các thành phần chính của NPM gồm:

- **Nginx:**
 - Thành phần reverse proxy chính.
 - Nhận request HTTP/HTTPS.
 - Chuyển tiếp request đến backend.
- **Giao diện quản trị web:**
 - Cho phép cấu hình proxy host, SSL, access list, user.
 - Mặc định chạy tại cổng 81.
- **Database:**
 - Lưu thông tin cấu hình.
 - Có thể dùng SQLite hoặc MariaDB/MySQL tùy cách triển khai.
- **Let's Encrypt:**
 - Tự động cấp chứng chỉ SSL miễn phí.
 - Hỗ trợ gia hạn chứng chỉ.

5.2.2 Cấu hình và quản trị với NPM

Hiện nay có nhiều reverse proxy phổ biến: Nginx, Traefik, Caddy, HAProxy... Mỗi phần mềm đều có những điểm mạnh, nhưng hầu hết yêu cầu bạn viết config file thủ công, dài dòng và dễ sai cú pháp.

Nginx Proxy Manager (NPM) giải quyết vấn đề đó bằng cách cung cấp một giao diện web để quản lý reverse proxy. Bạn không cần viết một dòng config Nginx nào, tất cả thao tác đều click trên trình duyệt.

Các lý do lựa chọn Nginx Proxy Manager:

- **GUI web-based:** thêm domain, cấu hình proxy, quản lý SSL bằng vài cú click
- **Tích hợp Let's Encrypt:** SSL miễn phí, tự động gia hạn, không cần chạy certbot thủ công

- **Access Lists:** giới hạn truy cập cho các trang admin
- **Custom Nginx config:** vẫn có thể thêm config Nginx tùy chỉnh khi cần
- **Chạy bằng Docker:** deploy trong 2 phút, không ảnh hưởng hệ thống host

Nói ngắn gọn: NPM biến việc quản lý reverse proxy từ “sửa config file, reload nginx, debug lỗi cú pháp” thành “điền form, bấm Save”.

Deploy Nginx Proxy Manager bằng Docker Compose

Để bắt đầu triển khai cài đặt và cấu hình NPM với docker compose, ta sẽ tạo thư mục để lưu trữ các tài nguyên của dự án:

```
$mkdir -p ~/nginx-proxy-manager  
$cd ~/nginx-proxy-manager
```

Tiếp theo sẽ tạo file `docker-compose.yml`:

```
services:  
  npm:  
    image: jc21/nginx-proxy-manager:latest  
    container_name: nginx-proxy-manager  
    restart: unless-stopped  
    ports:  
      - "80:80"      # HTTP  
      - "443:443"    # HTTPS  
      - "81:81"      # Admin panel  
    volumes:  
      - npm_data:/data  
      - npm_letsencrypt:/etc/letsencrypt  
    networks:  
      - proxy-network  
  
networks:  
  proxy-network:  
    name: proxy-network  
    driver: bridge  
  
volumes:  
  npm_data:  
  npm_letsencrypt:
```

Giải thích cấu hình:

- **Port 80:** nhận traffic HTTP (sẽ redirect sang HTTPS)
- **Port 443:** nhận traffic HTTPS
- **Port 81:** giao diện admin của NPM: chỉ dùng để quản lý, sau khi setup xong có thể đóng port này trên firewall
- **npm_data:** lưu database (SQLite mặc định), cấu hình proxy hosts
- **npm_letsencrypt:** lưu SSL certificates từ Let's Encrypt
- **proxy-network:** network dùng chung giữa NPM và các container app: quan trọng nhất, đừng quên!

Trong trường hợp khi bị lỗi xung đột port 80 hoặc 443 trên máy ảo, hãy đổi cổng dịch vụ sang 8080 hoặc 8443 để khắc phục lỗi này.

Sau khi thiết lập xong, khởi động container bằng câu lệnh:


```
$docker compose up -d
```

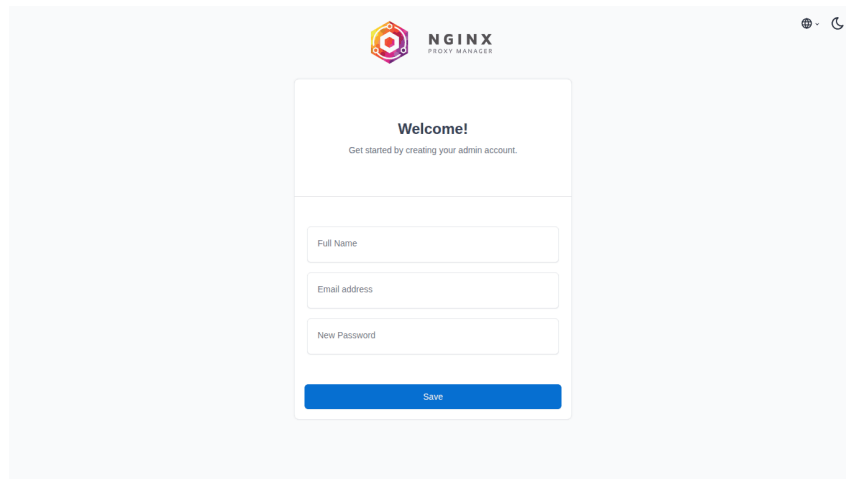
Kết quả sẽ hiển thị thấy container đang hoạt động và các cổng đang được lắng nghe theo đúng thiết lập:

```
anhchuyen@linuxserver: /npm$ docker compose ps
NAME                IMAGE                                COMMAND                  SERVICE    CREATED        STATUS        PORTS
npm-app-1           jc21/nginx-proxy-manager:latest     "/init"                 app        9 minutes ago  Up 9 minutes  0.0.0.0:81->81/tcp
p, [::]:81->81/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
anhchuyen@linuxserver: /npm$
```

Hình 5.6: Container Nginx Proxy Manager đang hoạt động

Tiếp theo có thể truy cập vào NPM qua địa chỉ `http://[IP]:81`, lưu ý cần thay địa chỉ IP cho phù hợp với cấu hình trên máy.

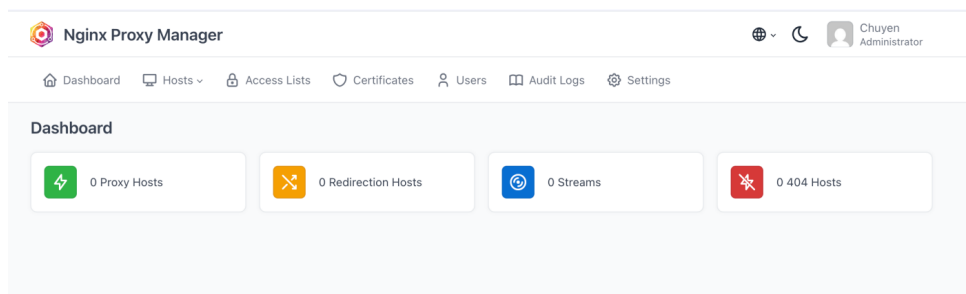
Giao diện đăng nhập vào hệ thống sẽ hiển thị ra như sau:



Hình 5.7: Giao diện đăng nhập Nginx Proxy Manager

Email đăng nhập: `admin@example.com`, Password: `changeme`

Thông tin này sẽ cần thay đổi sau khi đăng nhập lần đầu thành công. Giao diện trang quản trị sẽ như sau:



Hình 5.8: Giao diện quản trị Nginx Proxy Manager

Để thử nghiệm chức năng proxy server của NPM, ta sẽ thiết lập 2 website hoạt động phía sau và được quản lý bởi proxy. Các website này sẽ được triển khai qua docker compose và điều quan trọng nhất là cần tích hợp vào sử dụng network với NPM. Ta sẽ tạo mới một network bằng lệnh:

```
$docker network create npm_network
```

Tạo website tĩnh HTML và lưu trong thư mục html như sau:

```
$mkdir -p ~/my-static-site/html  
$cd ~/my-static-site
```

Tạo file `index.html` có nội dung bất kỳ lưu trong thư mục html vừa tạo ở trên.
Tiếp theo, tạo file `docker-compose.yml` cho website tĩnh này với nội dung:

```
services:  
  static-site:  
    image: nginx:alpine  
    container_name: static-site  
    restart: unless-stopped  
    networks:  
      - npm_network  
    volumes:  
      - ./html:/usr/share/nginx/html:ro  
  
networks:  
  npm_network:  
    external: true
```

Khởi động bằng câu lệnh:

```
$docker compose up -d
```

Tiếp theo, ta cần thiết lập cho website Wordpress hoạt động theo cách tương tự, tạo thư mục để lưu tài nguyên:

```
$mkdir -p ~/my-wordpress  
$cd ~/my-wordpress
```

Để lưu thông tin bí mật của CSDL, ta sẽ tạo file `.env` với nội dung như sau:

```
MYSQL_ROOT_PASSWORD=YourStrongRootPass123  
MYSQL_DATABASE=wordpress  
MYSQL_USER=wpuser  
MYSQL_PASSWORD=YourStrongWpPass123
```

Nội dung của file `docker-compose.yml` sẽ như sau:

```
services:  
  db:  
    image: mariadb:10.5  
    container_name: wp-db  
    restart: unless-stopped
```

```
env_file: .env
environment:
  MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  MYSQL_DATABASE: ${MYSQL_DATABASE}
  MYSQL_USER: ${MYSQL_USER}
  MYSQL_PASSWORD: ${MYSQL_PASSWORD}
volumes:
  - db_data:/var/lib/mysql
networks:
  - npm_network

wordpress:
  image: wordpress:latest
  container_name: wordpress
  restart: unless-stopped
  depends_on:
    - db
  env_file: .env
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: ${MYSQL_USER}
    WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
    WORDPRESS_DB_NAME: ${MYSQL_DATABASE}
  volumes:
    - wp_data:/var/www/html
  networks:
    - npm_network
  ports:
    - "8080:80"

volumes:
  db_data:
  wp_data:

networks:
  npm_network:
    external: true
```

Khởi động Wordpress container bằng câu lệnh:

```
$docker compose up -d
```

Chờ 1–2 phút rồi truy cập tạm thời `http://IP_server:8080` (hoặc port bạn map nếu có) để hoàn tất cài đặt WordPress lần đầu.

Như vậy chúng ta đã thiết lập xong 2 website: 1 web HTML tĩnh và 1 web Wordpress chạy trên docker compose và sử dụng chung network với NPM. Tiếp theo sẽ cần cấu hình proxy host trong NPM.

Truy cập NPM: `http://[IP]:81`

Truy cập vào Hosts → Proxy Hosts → Add Proxy Host

Proxy cho Website HTML (Static)

- Domain Names: `static.example.com` (thay bằng domain thật)
- Scheme: `http`
- Forward Hostname / IP: `static-site` (tên container)

- Forward Port: 80
- Cache Assets: Bật
- Block Common Exploits: Bật
- Websockets Support: Không cần

Tại tab SSL:

- Request a new SSL certificate
- Force SSL: Bật
- Nhấn Save

Cấu hình proxy cho Website WordPress:

- Domain Names: `blog.example.com` (hoặc `wp.example.com`)
- Scheme: `http`
- Forward Hostname / IP: `wordpress` (tên container)
- Forward Port: 80
- Block Common Exploits: Bật

Tab SSL: Tương tự, request SSL + Force SSL.

Quan trọng cho WordPress (sau khi tạo Proxy Host): Vào Advanced tab của Proxy Host WordPress, dán đoạn config sau:

```
location / {
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# WordPress specific
client_max_body_size 100m;
```

Sau đó vào WordPress Admin → Settings → General, sửa:

- WordPress Address (URL): `https://blog.example.com`
- Site Address (URL): `https://blog.example.com`

Lưu lại và clear cache nếu có.